

ProofForge: A Lean-First Multi-Chain Smart Contract Compilation Platform with Compile-Time Capability Verification

DaviRain Su
davirain.dev@gmail.com
Independent Researcher
China

ABSTRACT

Smart contract development across heterogeneous blockchain platforms (EVM, Solana, NEAR, Move-based chains, ZK circuits) remains fragmented: each ecosystem demands its own language, toolchain, and security audit. We present **ProofForge**, a Lean-first multi-chain smart contract compilation platform that enables developers to write contract business logic, state-machine rules, and formal proofs once in Lean 4, then compile to chain-native artifacts across multiple target families through a single portable intermediate representation (IR). The platform’s core innovation is a *capability-based routing model*: each target declares which capabilities it supports, and the compiler rejects unsupported contract intents at compile time rather than silently changing semantics. We describe the four-layer architecture (authoring SDK → ContractSpec → portable IR → target backends), the capability registry with 23 registered capability identifiers across 9 target profiles, and three production-grade backends—EVM (via Yul/solc), Solana sBPF (direct assembly), and NEAR/Wasm (via EmitWat). The platform includes executable IR semantics with decide-checked trace obligations, ownership rules for heap-backed locals, and a staged formal verification roadmap (FV-1 through FV-8). The unified testkit achieves behavior parity and resource budget baselines for the Counter and ValueVault shared scenarios across all three primary targets. Gate P0—the primary-chain completion covenant—is closed as of July 2026.

CCS CONCEPTS

• **Software and its engineering** → **Compilers; Software verification and validation**; • **Security and privacy** → *Distributed systems security*.

KEYWORDS

smart contracts, multi-chain compilation, Lean 4, portable intermediate representation, capability routing, formal verification, EVM, Solana, NEAR, WebAssembly

ACM Reference Format:

DaviRain Su. 2026. ProofForge: A Lean-First Multi-Chain Smart Contract Compilation Platform with Compile-Time Capability Verification. In *Proceedings of International Conference on AI and Blockchain Technology (AIBT 2026)*. ACM, New York, NY, USA, 8 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 INTRODUCTION

The blockchain ecosystem has fragmented into isolated platform silos—Ethereum (EVM), Solana (sBPF), NEAR (Wasm), Aptos/Sui (Move), and ZK circuit targets—each with its own programming language, virtual machine, storage model, and toolchain. A contract deployed on Ethereum cannot run on Solana; a Solana program cannot execute on NEAR. Cross-chain deployment requires complete rewrites, duplicate security audits, and platform-specific testing, leading to duplicated effort and inconsistent guarantees.

Existing multi-chain approaches attempt to solve this through either (a) a “lowest common denominator” language that abstracts away platform differences [9] or (b) transpilation between platform-native languages [11]. Both approaches have fundamental limitations: the former cannot express platform-specific features (Solana PDAs, EVM delegate calls, Move resources), and the latter risks semantic drift during translation. Neither approach integrates formal verification into the compilation pipeline.

We present **ProofForge** [12], a Lean-first multi-chain smart contract compilation platform built on three design principles:

- (1) **One verified source, multiple targets.** Contract logic, state-machine rules, and invariants are written once in Lean 4 [4]. The compiler lowers them through a portable intermediate representation to chain-native artifacts.
- (2) **Reject, don’t silently change semantics.** A capability-based routing model ensures that if a contract uses an operation unsupported by the selected target, the compiler emits a diagnostic with the specific capability identifier and target identifier—rather than producing incorrect code.
- (3) **Proofs gate codegen.** Formal verification is integrated into the pipeline: proofs are checked in Lean before backend lowering, not retrofitted after deployment.

ProofForge currently supports three production-grade backends: EVM (via Yul and solc [14]), Solana (via direct sBPF assembly to ELF [10]), and NEAR (via EmitWat to Wasm [7, 13]). Additional backends for Aleo Leo (ZK [1]), Cloudflare Workers (off-chain), and Move (Aptos [2]) are in research or frozen-spike stages. The platform has closed Gate P0—the primary-chain completion covenant—meaning all three primary targets pass behavior parity and resource budget tests for the shared Counter and ValueVault scenarios. Figure 1 shows the overall compilation pipeline.

This paper makes the following contributions:

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

AIBT 2026, July 3–5, 2026, Shanghai, China

© 2026 ACM.

ACM ISBN 978-1-4503-XXXX-X/26/07

<https://doi.org/XXXXXXX.XXXXXXX>

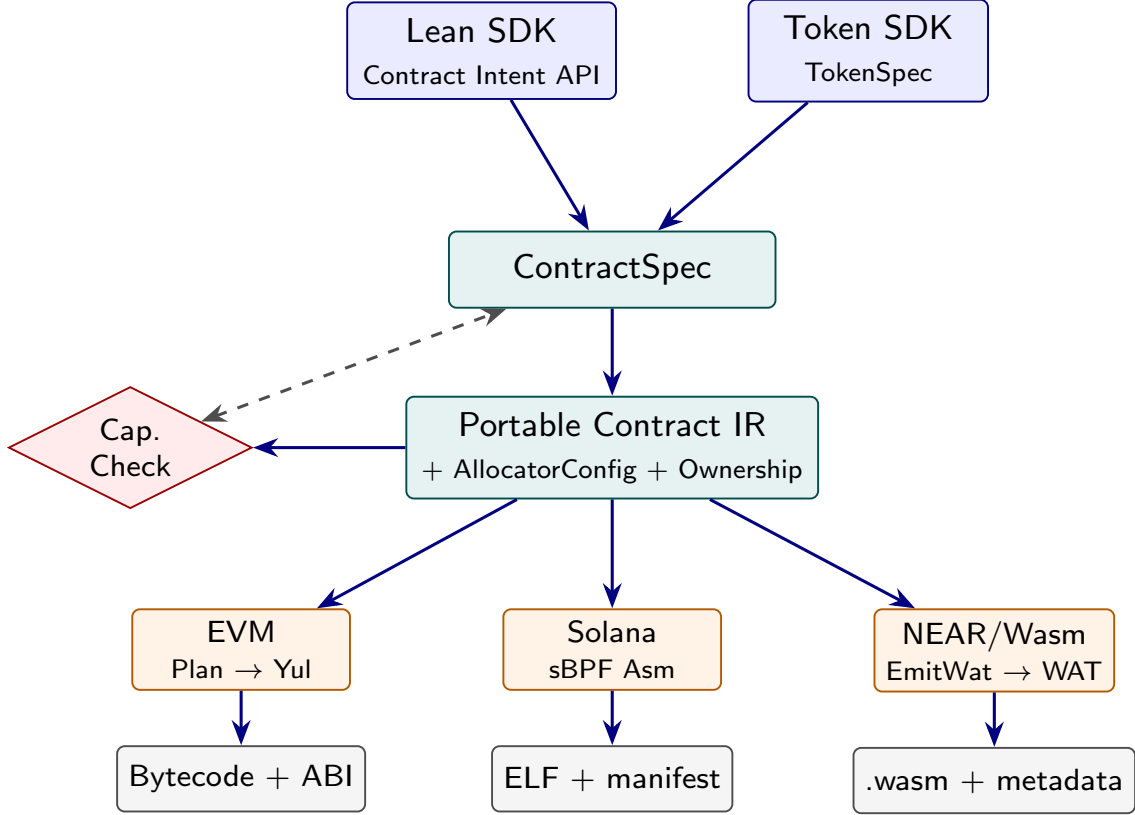


Figure 1: ProofForge compilation pipeline. Lean source lowers through **ContractSpec** to a portable IR, which is routed via capability checks to target-specific backends. Unsupported capabilities are rejected before artifact generation.

- A **portable contract IR** with typed effects, ownership rules, and capability annotations that sits between chain-neutral contract logic and target-specific backends (§3).
- A **capability routing model** with 23 registered capability identifiers across 9 target profiles, enabling compile-time rejection of unsupported operations (§4).
- Three **production-grade backend implementations**—EVM/Yul, Solana/sBPF-asm, and NEAR/EmitWat—each with full artifact emission, deployment metadata, and validation gates (§5).
- An **executable IR semantics** with decide-checked trace obligations and a staged formal verification roadmap (§6).
- A **unified testkit** achieving behavior parity and resource budget baselines for shared scenarios across three blockchain target families (§7).

2 BACKGROUND AND MOTIVATION

2.1 The Multi-Chain Problem

Smart contract platforms differ along multiple dimensions that make cross-chain compilation non-trivial. Table 1 summarizes the key semantic differences.

These differences mean a naive “write once, run everywhere” approach either (a) restricts contracts to the intersection of all platforms, losing platform-specific capabilities, or (b) silently maps

operations to incorrect semantics, introducing security vulnerabilities. For example, Solana’s Program-Derived Addresses (PDAs) and Cross-Program Invocations (CPI) have no direct EVM analog, while EVM’s DELEGATECALL has no Solana equivalent.

2.2 Lean 4 as a Contract Language

Lean 4 [4] is a purely functional programming language with dependent types and a built-in theorem prover. Its key properties make it suitable for smart contract development:

- **Dependent types** enable expressing invariants like “the sum of balances equals the total supply” at the type level.
- **Proof terms** can be constructed and checked before code generation, ensuring that verified properties hold at runtime.
- **LCNF** (Lean Compiler Normal Form) provides a CPS-style intermediate representation that ProofForge lowers through EmitYul for EVM targets.
- **Metaprogramming** allows the Contract Intent API to be embedded as a domain-specific language within Lean itself.

2.3 Related Work

Reach [9] is the closest precedent: a domain-specific language for writing multi-chain contracts. However, Reach does not integrate formal proofs, and its semantics are determined at the language

Table 1: Key semantic differences across blockchain target families.

Dimension	EVM	Solana	NEAR	Move
State model	Global slot storage	Account data	Key-value store	Resources / objects
Execution model	Stack VM	sBPF register VM	Wasm host	Move VM
Cross-contract call	CALL/DELEGATECALL	CPI with account metas	Async promises	Module calls
Identity	msg.sender	Signer accounts	Predecessor account	Transaction sender
Value transfer	msg.value (wei)	SOL via system program	attached deposit	Module resources
Resource budget	Gas	Compute units (CU)	Gas (Tgas)	Gas
Heap management	None (scratch memory)	Bump allocator	Host-managed	VM-managed

level rather than through a capability model. **Solang** [11] compiles Solidity to non-EVM targets (Solana, Polkadot) but starts from an EVM-centric source language and cannot express platform-native patterns. **Move** [2] provides resource-oriented programming for Aptos/Sui but is tied to the Move VM. **Certora** [6] provides formal verification for EVM contracts but targets a single chain. No existing platform combines Lean 4’s proof capabilities with multi-chain compilation through a capability-routed portable IR.

3 ARCHITECTURE

ProofForge’s architecture follows a four-layer model, designed to separate concerns between contract logic, target routing, and artifact generation.

3.1 Layer 1: Authoring SDK (Contract Intent API)

The default SDK surface is the **Contract Intent API**, a chain-neutral interface that exposes state declarations, entrypoints, events, caller/value access, checked arithmetic, assertions, and proofs—without importing any destination-chain module. The API deliberately does not expose EVM storage slots, Solana account metas, Move resources, or Wasm host ABIs.

For contracts that need chain-specific semantics (e.g., Solana PDA/CPI), **Target Extension SDKs** provide explicit chain-native operations. These extensions lower through capability identifiers and target metadata rather than adding chain-only constructors to the portable IR (Decision D-027), ensuring the IR remains chain-neutral.

The Contract Intent API also includes a multi-chain **Token SDK** (TokenSpec) that routes one token intent to ERC-20 bytecode on EVM or SPL Token / Token-2022 deployment plans on Solana.

3.2 Layer 2: ContractSpec

The ContractSpec is the compiler-owned intermediate representation produced from the authoring SDK. It captures the contract’s declared entrypoints, state, events, and invariants without any target-specific information. The spec is the input to the portable IR extractor and the capability resolution phase.

3.3 Layer 3: Portable Contract IR

The portable IR is the central abstraction of the platform. It sits between the chain-neutral ContractSpec and target-specific backends, expressing chain-independent business logic plus target-resolved capability effects. The IR is defined in ProofForge. IR.Contract as a set of Lean 4 inductive types. Figure 2 shows the IR module structure and its relationship to the compilation pipeline.

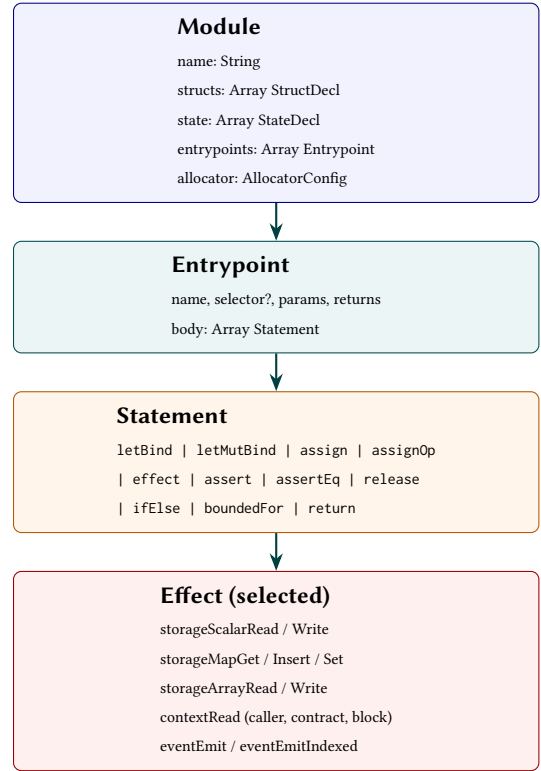


Figure 2: Portable IR module structure. A Module contains state declarations and typed entrypoints. Each entrypoint body is a sequence of statements, which may produce typed effects that carry capability identifiers.

Each effect node carries a capability identifier derivable from the effect constructor via the `Effect.capability` function. This mapping is the basis for compile-time capability checking: the target adapter inspects the capabilities used by the IR and rejects the build if any are unsupported. The key IR types are:

```

inductive ValueType where
| unit | bool | u32 | u64 | hash
| fixedArray (element : ValueType) (length : Nat)
| structType (name : String)

inductive Statement where
| letBind (name : String) (type : ValueType) (value : Expr)
| letMutBind (name : String) (type : ValueType) (value : Expr)
| assign (target value : Expr)
| assignOp (target : Expr) (op : AssignOp) (value : Expr)
| effect (effect : Effect)
| assert (condition : Expr) (message : String)
| release (name : String) -- release owned local
  
```

```

| ifElse (condition : Expr) (thenBody elseBody : Array
  Statement)
| boundedFor (indexName : String) (start stopExclusive : Nat) (
  body : Array Statement)
| return (value : Expr)

structure Module where
  name : String
  structs : Array StructDecl := #[]
  state : Array StateDecl
  entrypoints : Array Entrypoint
  allocator : AllocatorConfig := defaultAllocator

```

The shared scenario Counter contract, expressed as portable IR, demonstrates how one module lowers to multiple targets without modification:

```

def module : Module := {
  name := "Counter"
  state := #[{ id := "count", kind := .scalar, type := .u64 }]
  entrypoints := #[
    { name := "initialize", returns := .unit,
      body := #[.effect (.storageScalarWrite "count" (.literal (.
        u64 0))) ] },
    { name := "increment", returns := .unit,
      body := #[
        .letBind "n" .u64 (.effect (.storageScalarRead "count")),
        .effect (.storageScalarWrite "count"
          (.add (.local "n") (.literal (.u64 1)))) ] },
    { name := "get", returns := .u64,
      body := #[.return (.effect (.storageScalarRead "count")) ]
      }
  ]
}

```

This same IR module lowers to EVM Yul (with storage slot mapping), Solana sBPF assembly (with account data read/write), and NEAR WAT (with key-value storage)—without any modification to the IR itself.

3.4 Layer 4: Target Backends

Each target backend is responsible for: (1) declaring a target profile (supported capabilities, allocator config, metadata); (2) lowering accepted IR constructs to target-native code; (3) rejecting unsupported constructs with explicit diagnostics; and (4) emitting artifacts (bytecode, ELF, Wasm) plus deployment manifests and ABI/IDL metadata. The target registry (ProofForge.Target.Registry) maintains profiles for each registered target id. The CLI selects the target via `--target <id>`, which activates the corresponding adapter and capability set.

4 CAPABILITY-BASED TARGET ROUTING

The capability model is ProofForge’s core design principle: *a contract that routes to a target only uses capabilities that target supports*. Unlike a monolithic IR where all constructs are available on all targets, ProofForge’s IR marks each effect with a capability identifier, and the target adapter checks support before lowering.

4.1 Capability Registry

The canonical capability registry defines 23 core capability identifiers across 13 domains. Table 2 shows the support matrix for primary and representative targets.

4.2 Capability Resolution

When the compiler receives a `--target` flag, it performs the resolution sequence shown in Figure 3.

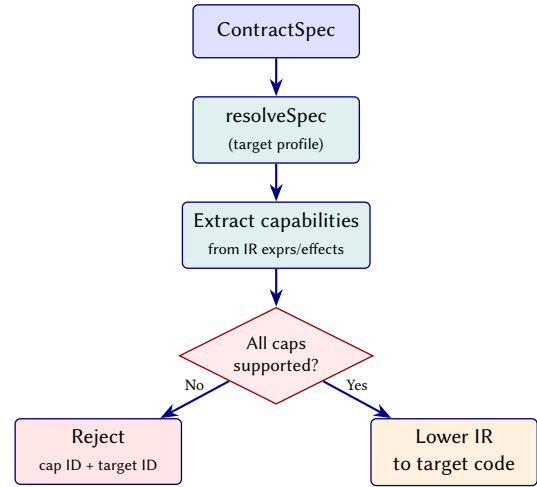


Figure 3: Capability resolution flow. The target adapter resolves the contract spec, extracts required capabilities from the IR, and checks them against the target profile. Unsupported capabilities produce a diagnostic; only fully-supported contracts proceed to lowering.

4.3 Ownership and Allocator Model

The IR includes an ownership system for heap-backed locals (the release statement). An ownership checker (ProofForge.IR.Ownership.lean) validates that released locals are never used afterward (no use-after-release) and never released twice (no double-release). Each target configures its allocator via `AllocatorConfig`; see Table 3.

The ownership checker ensures that the three different lowerings of release (EVM rejects it, Solana lowers to a no-op, NEAR lowers to `memory.free`) are safe only if the IR-level discipline is proven (FV-3 target).

5 BACKEND IMPLEMENTATIONS

5.1 EVM Backend (Baseline)

The EVM backend is the most mature target. The pipeline:

```

Lean contract source
-> Lean frontend / LCNF
-> EmitYul (LCNF -> Yul AST)
-> Yul source
-> solc --strict-assembly
-> EVM runtime bytecode
-> Foundry smoke tests

```

For portable IR contracts, the EVM backend introduces a **semantic plan layer** between the IR and Yul generation (RFC 0004). The plan defines `ExprPlan`, `StmntPlan`, `EntrypointPlan`, `EventPlan`, `CrosscallPlan`, and `MetadataPlan`, which are validated and type-inferred before Yul generation. This separation ensures that the Yul output is type-correct and that checked arithmetic semantics (trap on overflow) are enforced at the plan level.

The EVM SDK (ProofForge.Evm) provides extern declarations for all EVM opcodes (`calldata.load`, `sstore`, `sload`, `caller`, `callvalue`, `keccak256`, `log0-log4`, etc.) that are recognized by `EmitYul` and lowered directly to EVM instructions. Typed storage helpers (`Storage.Var`,

Table 2: Core capability registry (selected). Y = supported, P = partial/spike, N = not supported.

Capability ID	EVM	NEAR	Solana	Psy	CF W.
storage.scalar	Y	Y	Y	Y	Y
storage.map	Y	Y	P	P	Y
storage.array	P	N	Y	P	P
caller.sender	Y	Y	Y	P	Y
value.native	Y	Y	Y	P	N
events.emit	Y	Y	Y	Y	Y
crosscall.invoke	Y	N	N	P	Y
crosscall.cpi	N	N	Y	N	N
storage.pda	N	N	Y	N	N
crypto.hash	Y	Y	Y	Y	Y
assertions.check	Y	Y	Y	P	Y
control.conditional	P	N	Y	Y	Y
control.bounded_loop	N	N	P	P	Y
zk.circuit	N	N	N	Y	N
runtime.allocator	N	Y	Y	P	N

Table 3: Allocator configuration per target.

Target	Strategy	Region	Release
EVM	Bump	Call-scratch memory	None (rejected)
Solana	Bump	Heap at 0x300000000	Noop
NEAR	Host	Wasm linear memory	Noop

Storage.Map, Storage.Map2) provide compile-time slot assignment and mapping slot computation via `keccak256(key || slot)`.

Validation gates include: 58 diagnostic cases, 99 IR coverage entries, 19 IR smoke tests, Foundry runtime smoke tests, and Anvil deploy tests [5].

5.2 Solana sBPF Assembly Backend

The Solana backend generates sBPF assembly directly from the portable IR, bypassing intermediate languages like Zig or Rust (Decision D-026). The pipeline:

```
portable IR
-> sBPF assembly (.s)
-> sbpf toolchain (assemble + link)
-> ELF (.so) program binary
-> Mollusk / Surfpool / Pinocchio tests
```

The backend defines Solana-specific constructs in `ProofForge.Solana`: `AccountMeta` (writable/read-only accounts with signer policies), `PdaBinding` (program-derived address derivation with seeds and bump), `CpiCall` (cross-program invocations with account metas and signer seeds), `SysvarReadAction` (system variable reads: rent, epoch schedule, etc.), `MemoryAction` (runtime memory operations: `memcpy`, `memset`, `memcmp`), `CryptoHashAction` (SHA-256, Keccak-256, Blake3), and `ComputeUnitsAction` (compute budget introspection).

These Solana-specific constructs are gated by `crosscall.cpi` and `storage.pda` capabilities (D-027) and never appear in the portable IR itself—they live in the Solana backend and SDK layer.

Validation includes Pinocchio [8] live dual-deploy equivalence tests across five scenarios (System transfer/create_account, SPL Token transfer/ops/authority), all passing with 5 passed, 0 skipped, 0 failed. The ELF compatibility blocker was resolved by forwarding `--solana-sbpf-arch v0`, producing loader-compatible v0 ELFs.

5.3 NEAR/Wasm Backend (EmitWat)

The NEAR backend uses `EmitWat`, which lowers portable IR to a Wasm AST and then to WAT (WebAssembly Text Format) text, compiled to `.wasm` via `wat2wasm` [13]. This approach mirrors the EVM’s portable-IR-to-Yul pattern:

```
portable IR
-> EmitWat (Wasm AST construction)
-> WAT text
-> wat2wasm
-> .wasm binary
-> offline host execution / NEAR deployment
```

`EmitWat` avoids the macro coupling of `Rust near-sdk-rs` source generation (D-031) and the `Lean-runtime-to-Wasm` port that blocked prior approaches. Because the portable IR already abstracts over Lean objects (only `u32/u64/bool/hash` scalars + storage effects), `EmitWat` needs no Lean runtime port, object-model boxing, or garbage collection.

The NEAR backend includes target-first check, emit, and build commands, writes `proof-forge-artifact.json` and `proof-forge-deploy.json`, validates WAT/Wasm hashes, ABI entrypoints, and capabilities, and executes the generated Counter WAT through an offline host.

5.4 Research and Frozen Backends

Additional backends exist in research or frozen-spike stages: **Psy/DPN** (portable IR → `.psy` → Dargo → DPN circuit JSON, experimental), **Aleo Leo** (portable IR → Leo package, research spike with golden fixtures [1]), **Cloudflare Workers** (portable IR → TypeScript Worker, off-chain host), **Move/Aptos** [2] (frozen at M1/M2 spike per D-044), and **CosmWasm** [3] (frozen at M1/M2 spike).

6 FORMAL VERIFICATION

ProofForge is Lean-first, but most current assurance comes from tests and golden files rather than theorems. The formal verification roadmap (FV-1 through FV-8) progressively turns test-validated invariants into machine-checked theorems. Proofs gate codegen, per RFC 0001: no proof transport to chains, no on-chain verification of Lean proofs.

6.1 Existing Formal Anchors

Three formal anchors are already implemented on main; see Table 4.

Table 4: Existing formal verification anchors.

Anchor	What it provides
IR Semantics (IR/Semantics.lean)	Trace interpreter for scalars, arrays, structs, storage, control flow, events; decide-checked
Ownership Rules (IR/Ownership.lean)	release/owned-local discipline; no use-after-release, no double-release
Trace Obligations (Backend/*/Refinement.lean)	decide-checked theorems: Counter, ValueVault + 8 probe traces match expected observable values on EVM Yul and NEAR WAT

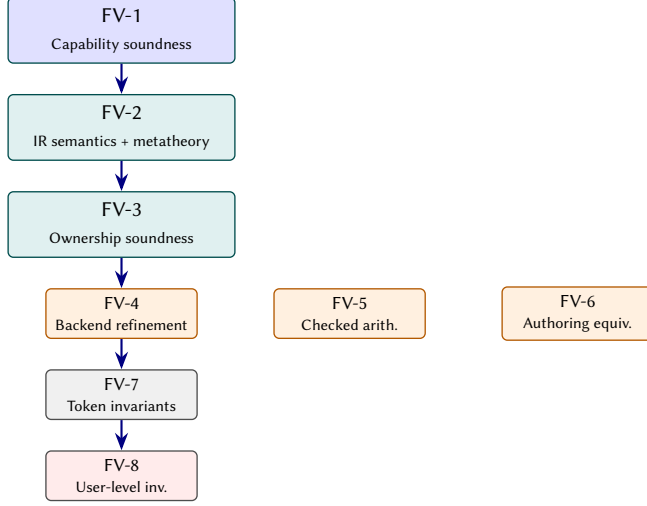


Figure 4: Formal verification roadmap. FV-1 (capability soundness) and FV-2 (IR semantics) are foundational. FV-3 depends on FV-2. FV-4 (backend refinement) builds on FV-3. FV-5/FV-6 stabilize target surfaces. FV-7/FV-8 are product-level proofs.

6.2 Verification Targets (Priority Order)

Figure 4 shows the staged FV roadmap and its dependency structure.

FV-1 (Capability Routing Soundness): Prove that if `resolveSpec` profile `spec` is `.ok` plan, then every capability in `plan.calls` is in `profile.capabilities` (soundness), and that if the spec references a capability outside the profile, `resolveSpec` returns `.error` (completeness of rejection). These are structural inductions over arrays—no semantics needed.

FV-2 (IR Semantics): Extend `IR/Semantics.lean` from the scalar subset toward the full checked IR surface. State and prove: determinism (one trace per input/state), progress and preservation for the typed subset, and bounded termination for `boundedFor` with static bounds. Keep the semantics executable (decide-friendly) so CI checks stay cheap.

FV-3 (Ownership/Release Soundness): Once FV-2 gives release-aware semantics, prove that programs accepted by the ownership checker never evaluate a released local and never release twice. This is the formal justification for lowering release to allocator frees in `EmitWat` while EVM/Psy reject it and TS ignores it.

FV-4 (Backend Refinement): Replicate the `TraceObligation` pattern per backend against the shared scenario (Counter first, ValueVault second). Each backend earns “Experimental $\rightarrow$ Supported” only with the scenario differential gate and at least the export/trace obligation theorem.

Table 5: Testkit behavior parity results (Gate G0). All three primary targets pass Counter, ValueVault (11 calls), and unsupported-capability diagnostic tests.

Scenario	EVM	Solana	NEAR
Counter trace	✓	✓	✓
ValueVault (11 calls)	✓	✓	✓
Unsupported cap. diag.	✓	✓	✓

FV-5–FV-8: Checked arithmetic semantics per target (FV-5), authoring-surface equivalence for `.learn` and `contract_source` (FV-6), token plan invariants (FV-7), and user-level contract invariants (FV-8)—the long-term differentiator that lets contract authors prove properties like *balance* = *deposits* – *releases* + *fees* before codegen.

7 EVALUATION

7.1 Shared Scenarios

ProofForge defines two shared scenarios for cross-target validation: **Counter** (a minimal stateful contract with `initialize`, `increment`, and `get`, testing scalar storage, arithmetic, and return values) and **ValueVault** (a complex contract with deposits, releases, fees, and balance invariants, testing events, access control, and aggregate state).

7.2 Behavior Parity

The unified testkit (RFC 0007) executes the shared scenarios across all three primary targets and verifies trace parity. Table 5 summarizes the results.

7.3 Resource Budgets

Per D-040, Tier-0 parity requires comparing behavior *and* resource budgets. The testkit scenarios pin budget baselines with tolerance bands for EVM gas, Solana compute units (CU), and NEAR gas (Tgas). Table 6 summarizes the validation coverage per target.

7.4 Gate P0 Closure

Gate P0—the primary-chain completion covenant (D-045)—was closed on July 4, 2026, at commit 466b320, after GitHub CI run 28677055773 completed successfully. The closing run validated all three primary targets including behavior parity, resource budgets, capability diagnostics, Foundry/Anvil gates, Mollusk/Surfpool gates, offline host execution, artifact/deploy metadata, and formal trace obligations.

7.5 Comparison with Existing Approaches

Table 7 compares ProofForge with the two closest multi-chain platforms.

Table 6: Validation gate coverage per primary target.

Gate	EVM	Solana sBPF	NEAR/Wasm
Diagnostic cases	58	—	—
IR coverage entries	99	—	—
IR smoke tests	19	—	—
Foundry runtime smoke	✓	—	—
Anvil deploy smoke	✓	—	—
Mollusk unit tests	—	✓	—
Surfpool/Web3.js live smokes	—	✓	—
Pinocchio equivalence gates	—	5 scenarios	—
Offline host smoke	—	—	✓
Formal trace obligations	decide-checked	—	decide-checked
Artifact / deploy metadata	✓	✓	✓
Capability diagnostics	✓	✓	✓

Table 7: Comparison with multi-chain contract platforms.

Feature	ProofForge	Reach	Solang
Source language	Lean 4	Reach DSL	Solidity
Formal proofs	Integrated (Lean)	Linear temporal logic	No
Capability model	23 IDs, 9 profiles	Network-specific	EVM-centric
Compile-time reject	Capability + target ID	Partial	No
Targets	EVM + Solana + NEAR	EVM + Algorand	Solana + Polkadot
Portable IR	Yes (typed effects)	Partial	No
Ownership system	IR-level checker	No	No
Trace obligations	decide-checked	No	No

8 DISCUSSION

8.1 Design Decisions

The platform’s architecture is guided by 45 settled design decisions (D-001 through D-045), each documented with rationale and date. Key decisions include: **D-027** (CPI/PDA effects stay in a Solana-specific layer, not the portable IR); **D-028** (user contracts target a chain-neutral Contract Intent API; the selected target resolves intents into capability plans); **D-031** (EmitWat is the canonical Wasm-family backend); and **D-045** (the three priority chains must be completed to production-grade quality before any additional chain advances beyond docs-only research).

8.2 Limitations

Current limitations include: the portable IR expression set is a contract-relevant subset of Lean/LCNF, not full Lean preservation—higher-order functions, closures, and unbounded loops are restricted or rejected on most targets. Formal verification is in early stages: FV-1 through FV-3 are planned but not yet implemented; only FV-2 (executable IR semantics) and FV-4 (trace obligations) have decide-checked theorems. The EVM backend has the most mature validation surface; Solana and NEAR have fewer dedicated gates. The CLI is mid-migration from legacy flags to `proof-forge build|emit|check --target`; legacy aliases remain for compatibility.

9 CONCLUSION AND FUTURE WORK

ProofForge demonstrates that a Lean-first, capability-routed, portable-IR-centered architecture can produce production-grade smart contract artifacts across heterogeneous blockchain platforms without silently changing semantics. The platform’s core promise—*one verified contract source, multiple chain-native artifacts, unsupported operations rejected at compile time*—is validated by the closure of

Gate P0, with behavior parity and resource budgets confirmed across EVM, Solana sBPF, and NEAR/Wasm.

The staged formal verification roadmap (FV-1 through FV-8) provides a clear path from the current test-validated invariants toward machine-checked theorems, starting with capability routing soundness (FV-1) and IR semantics metatheory (FV-2), culminating in user-level contract invariant proofs (FV-8)—the long-term differentiator that lets contract authors prove properties like *balance = deposits – releases + fees* before codegen.

Future work follows the tiered target portfolio (D-034): **Tier-1**: advance CosmWasm and Move/Aptos from frozen spikes to M3/M4 after CLI migration completes. **Tier-2**: sourcegen targets (Stellar Soroban, Starknet Cairo, TON TVM, Cardano Plutus/Aiken, Tezos Michelson/LIGO) after two Tier-1 targets reach Experimental. **Policy family**: Bitcoin Script/Miniscript, Bitcoin Cash CashScript, Zcash Shielded, and Kaspas Toccata as spending-policy generators with a separate policy IR. **Cloud platform**: hosted target-matrix builds, artifact registry, testnet deployment, and verification reports.

ACKNOWLEDGMENTS

The ProofForge project builds on the Lean 4 theorem prover [4], the Foundry development toolkit [5], the Solana sBPF toolchain [10], and the NEAR/Wasm ecosystem [7]. The design was shaped by 45 settled design decisions reviewed through a structured RFC process.

REFERENCES

- [1] aleo 2024. Aleo: Zero-Knowledge Application Platform. <https://aleo.org/>.
- [2] Sam Blackshear, Evan Cheng, David L. Dill, Victor Gao, Todd Maurer, Todd Nowacki, Alistair Pott, Shaz Qadeer, Rain, Dario M. Russi, Stephane Sezer, Tim Zakian, and Libra Association. 2019. Move: A Language with Programmable Resources. In *Libra Blockchain*.
- [3] cosmewasm 2024. CosmWasm: Smart Contracts for the Cosmos Ecosystem. <https://cosmwasm.cosmos.network/>.
- [4] Leonardo de Moura and Sebastian Ullrich. 2021. The Lean 4 Theorem Prover and Programming Language. In *Proceedings of the 14th International Conference on Automated Deduction (CADE)*.

- [5] foundry 2024. Foundry: Smart Contract Development Toolkit. <https://book.getfoundry.sh/>.
- [6] Certora Ltd. 2023. Certora Prover: Formal Verification of Smart Contracts. <https://www.certora.com/>.
- [7] near 2024. NEAR Smart Contract Documentation. <https://docs.near.org/smart-contracts/what-is>.
- [8] pinocchio 2024. Pinocchio: Solana Program Development Toolkit. <https://github.com/pinocchio-labs/pinocchio>.
- [9] reach 2024. Reach: The Simplest Way to Build Decentralized Applications. <https://www.reach.sh/>. Accessed: 2026-07-03.
- [10] solana 2024. Solana Program Documentation. <https://solana.com/docs/core/programs>.
- [11] solang 2024. Solang: A Solidity Compiler for Solana and Polkadot. <https://solang.readthedocs.io/>. Accessed: 2026-07-03.
- [12] DaviRain Su. 2026. ProofForge: Lean-First Multi-Chain Smart Contract Platform. <https://github.com/DaviRain-Su/proof-forge>.
- [13] wasm 2024. WebAssembly Specification. <https://webassembly.org/>.
- [14] yul 2024. Yul: Intermediate Language for the Ethereum Virtual Machine. <https://docs.soliditylang.org/en/latest/yul.html>.